TECHNICAL REFERENCE & CURRICULUM GUIDE

Module 2: Transformer Architecture & Attention Mechanics

Module 2 deconstructs the core engine of modern Large Language Models: the Transformer architecture and its Attention mechanics. Every mathematical operation, tensor dimension, hardware bottleneck, and scaling algorithm is unpacked below from first principles.

2.1 Self-Attention Mechanism: Query, Key, Value & Scaled Dot-Product Walkthrough

Prior to the Transformer (Vaswani et al., 2017), sequence modeling relied on Recurrent Neural Networks (RNNs, LSTMs, GRUs) which processed tokens sequentially ($h_t = f(h_{t-1}, x_t)$). This created two fatal bottlenecks: **zero parallelization across sequence length during training** and **vanishing/exploding gradients over long distances**. **Self-Attention** reduces the maximum path length between any two tokens in a sequence to ***O(1)***, enabling full parallelization during training while dynamically routing information based on context.

1. The Information Retrieval Analogy & Linear Projections

Given an input sequence of N tokens embedded into a continuous vector space of dimension d_{model}, we represent the input as a matrix $X \in \mathbb{R}^{N \times d_{model}}$. Self-attention projects X into three distinct latent subspaces using three learnable weight matrices:

$$Q = X\,W_Q, \quad K = X\,W_K, \quad V = X\,W_V$$

where $W_Q \in \mathbb{R}^{d_{model} \times d_k}$, $W_K \in \mathbb{R}^{d_{model} \times d_k}$, and $W_V \in \mathbb{R}^{d_{model} \times d_v}$.

- **Query ($Q \in \mathbb{R}^{N \times d_k}$):** Represents *"what information this token is actively looking for."* (Analogy: The search query you type into a database engine).
- **Key ($K \in \mathbb{R}^{N \times d_k}$):** Represents *"what information or label this token advertises to other tokens."* (Analogy: The indexed metadata/title of each database record).
- **Value ($V \in \mathbb{R}^{N \times d_v}$):** Represents *"the actual semantic content payload this token passes forward if attended to."* (Analogy: The actual body content of the matched database record).

2. The Scaled Dot-Product Attention Formula — Step-by-Step Walkthrough

The complete equation for Scaled Dot-Product Attention is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q\,K^T}{\sqrt{d_k}} + M\right) V$$

1. **Step 1: Raw Similarity Scoring ($S = Q K^T$)**
   We compute the matrix product between Queries Q ($N \times d_k$) and transposed Keys K^T ($d_k \times N$), yielding a raw pairwise affinity matrix $S \in \mathbb{R}^{N \times N}$. Each scalar entry $S_{i,j} = q_i \cdot k_j = \sum_{m=1}^{d_k} q_{i,m} k_{j,m}$ measures unnormalized dot-product similarity.
2. **Step 2: Scaling by $1 / \sqrt{d_k}$ (Variance Stabilization Proof)**
   Assuming components of q and k have mean 0 and variance 1, the dot product sums d_k independent products, causing variance to grow linearly with d_k ($\text{StdDev} = \sqrt{d_k}$). Without scaling, large logits push Softmax into saturation regions where gradients vanish ($\frac{\partial p_i}{\partial z_j} \to 0$). Dividing by $\sqrt{d_k}$ forces variance back to exactly 1.
3. **Step 3: Masking ($+ M$)**
   Padding masks set padded positions to $-\infty$. Causal/Look-Ahead masks for autoregressive decoders set upper-triangular positions above the diagonal to $-\infty$ so future tokens receive an exact 0.0 weight after Softmax.
4. **Step 4: Row-Wise Softmax Normalization ($A = \text{softmax}(\cdot)$)**
   Converts raw logits into a valid probability distribution $A \in \mathbb{R}^{N \times N}$ where rows sum to 1.
5. **Step 5: Weighted Value Aggregation ($O = A V$)**
   Multiplies attention weights A by V, producing output $O \in \mathbb{R}^{N \times d_v}$ where each row is a convex combination of Value vectors.

Computational Complexity: $O(N^2 \cdot d_k)$ FLOPs and $O(N^2)$ memory footprint (mitigated in hardware by FlashAttention-2/3 via SRAM tiling and online Softmax scaling).

2.2 Multi-Head Attention (MHA) — Why Multiple Heads & What They Learn

If we compute only a single attention distribution over d_{model}, Softmax becomes dominated by a single dominant relationship, washing out secondary semantic and syntactic connections.

- **Mathematical Formulation:** Splits d_{model} into h parallel heads of dimension $d_k = d_v = d_{model} / h$:

  $$\text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W_O$$

  Each head computes attention independently in its subspace, and the concatenated output is linearly projected via W_O. Total FLOP cost equals a single full-rank attention pass.
- **What Individual Attention Heads Learn:** Mechanistic interpretability reveals heads self-organize into specialized circuits:

- 1. **Positional / Previous-Token Heads:** Attend strictly to position *t−1* or *t+1*.
  2. **Syntactic & Dependency Heads:** Connect verbs to direct objects or auxiliary verbs across long spans.
  3. **Coreference Resolution Heads:** Connect pronouns (*"it"*, *"they"*) to antecedent nouns.
  4. **Induction Heads:** Two-layer circuits that copy pattern completions (*[A][B] \dots [A] \to [B]*), forming the mechanistic basis for in-context few-shot learning.
  5. **Attention Sinks:** Dump residual Softmax mass onto the `` token when no relevant token matches.

2.3 Positional Encoding: Sinusoidal vs. Learned vs. RoPE vs. ALiBi

Because self-attention is permutation equivariant, positional encodings must be injected to preserve token order.

- **Absolute Sinusoidal (Vaswani, 2017):** Adds non-learned geometric sine/cosine waves of varying wavelengths directly to input embeddings. Extrapolation beyond training length L_{train} fails.
- **Learned Absolute (GPT-1/2/3, BERT):** Learnable lookup table $W_{pos} \in \mathbb{R}^{L_{max} \times d_{model}}$ added to embeddings. Zero extrapolation capability.
- **Rotary Position Embeddings — RoPE (Su et al., 2021 — LLaMA, Mistral, Qwen, Claude, Gemini):** Multiplies Query and Key vectors by rotation matrices $R_{\Theta,\, m}$ inside every attention layer. Because $\langle R_m\, q,\, R_n\, k \rangle = q^T\, R_{n-m}\, k$, the dot product depends *strictly on relative distance* $m - n$. Preserves vector norms and integrates natively with FlashAttention.
- **ALiBi — Attention with Linear Biases (Press et al., 2021):** Adds zero position embeddings; subtracts a static linear penalty $-\, m \cdot |i - j|$ directly from attention logits. Offers strong local extrapolation but weaker long-range retrieval than RoPE.

Encoding Scheme	Injection Point	Position Type	Extrapolation	FlashFriendly	Primary Models
Sinusoidal	Input Embeddings	Absolute	Poor	Yes	Original 2017 Transformer
Learned Absolute	Input Embeddings	Strictly Absolute	Impossible	Yes	BERT, GPT-1/2/3
ALiBi	Attention Logits	Relative Penalty	Excellent	Custom Kernel	BLOOM, MPT, Falcon
RoPE	Every Layer (Q, K)	Relative Rotation	Excellent (via Scaling)	Yes	LLaMA 1–4, Qwen, Gemini, Claude

2.4 Decoder-Only vs. Encoder-Decoder vs. Encoder-Only — When Each Is Used Today

- **Encoder-Only (BERT, ModernBERT, Embeddings):** Full bidirectional attention. Used for dense vector retrieval, semantic search, Rerankers, and high-throughput PII/classification classifiers.
- **Encoder-Decoder (T5, Whisper):** Bidirectional encoder + cross-attention decoder. Used for speech-to-text (Whisper) and specialized translation. Lost general LLM preference due to parameter splitting and KV management overhead.
- **Decoder-Only (GPT-4o/5, LLaMA, Claude, Gemini):** Causal attention stack. Won the AI landscape due to 100% pre-training compute utilization (every token is a prediction target), unified parameter sharing, and seamless multi-turn KV caching.

2.5 Context Window Extension: How 8K Becomes 128K to 1M+

To extend context without retraining from scratch, models modify RoPE frequencies:

- **Linear Position Interpolation (PI):** Divides positions by $s = L_{target} / L_{train}$. Squishes high-frequency dimensions, crowding token order and degrading local syntax.
- **NTK-Aware Scaling:** Increases the RoPE base b to $b' = b \cdot s^{d/(d-2)}$. Preserves high-frequency local token resolution while absorbing interpolation across low-frequency dimensions.
- **YaRN (Yet another RoPE extensioN):** Partitions RoPE dimensions into high-frequency (extrapolated without change), low-frequency (linearly interpolated), and medium-frequency bands, combined with Softmax logit temperature scaling ($1/\sqrt{t}$) to maintain attention entropy across 128K+ tokens.
- **Distributed Infrastructure:** Utilizes Ring Attention to shard massive KV caches across multiple GPUs in a ring topology.

2.6 KV-Cache Mechanics and Its Role in Inference Cost & Latency

During autoregressive generation, caching historical Key (K) and Value (V) tensors in VRAM avoids recomputing past tokens, reducing generation complexity from $O(T^3)$ to $O(T^2)$.

- **Prefill vs. Decode Phase:** Prefill processes prompt tokens in parallel via GEMM (Compute-Bound). Decode generates 1 token at a time via GEMV, loading model weights and the entire KV cache from HBM for every single token (**Memory-Bandwidth-Bound**).
- **VRAM Calculation:** For a 70B model ($n_{layers} = 80, d_{model} = 8192, n_{heads} = 64$) in FP16 at 128K context, the KV cache alone requires 320 GB of VRAM for a single user ($B = 1$), exceeding the 140GB model weight footprint. Modern serving engines (vLLM) solve this via PagedAttention and Prefix Caching.

2.7 Multi-Query Attention (MQA) & Grouped-Query Attention (GQA)

To slash KV-cache memory consumption and accelerate decode throughput, modern models share Key/Value heads across Query groups:

Attention Head Sharing Architectures:

- **MHA (Multi-Head Attention):** $n_{kv} = n_q$ (Dedicated K/V head per Query head; maximum VRAM footprint).
- **MQA (Multi-Query Attention):** $n_{kv} = 1$ (All Query heads share a single global K/V head; maximum memory savings, slight reasoning degradation).
- **GQA (Grouped-Query Attention):** $1 < n_{kv} < n_q$ (Queries divided into G groups sharing 1 K/V head per group; e.g., LLaMA-3 uses GQA-8 with 8x KV memory savings and virtually lossless reasoning).

Metric	Multi-Head (MHA)	Grouped-Query (GQA-8)	Multi-Query (MQA)
Query Heads (n_q)	64	64	64
KV Heads (n_{kv})	64 (1:1 per Query)	8 (1 per 8 Queries)	1 (Shared globally)
KV Cache Size (70B @ 128K)	320 GB	40 GB (8× savings)	5 GB (64× savings)
Decode Bandwidth Speedup	1× (Baseline)	~6× – 8× higher throughput	~10×+ higher throughput